

Hermes Original vs Hermes Turbo vs OpenClaw

Generated 2026-05-22 15:32 UTC • Side-by-side comparison across 9 categories. Lower is better unless explicitly marked as throughput. Historical Hermes Original / OpenClaw figures from the README v0.14.2 baseline; Hermes Turbo numbers from the new turbo benchmark suite (May 2026).

Headline scoreboard

Contender	Total Score	Strengths	Weak spots
Hermes Original	30 / 50	Memory, disk footprint, cold start	JSON, message pipeline, tool-call parse
Hermes Turbo	44 / 50	JSON, message pipeline, tool-call, ecosystem, cold start	Token counting, integrations breadth
OpenClaw	36 / 50	Concurrency / async, integrations breadth	Memory (RSS), disk, cold start

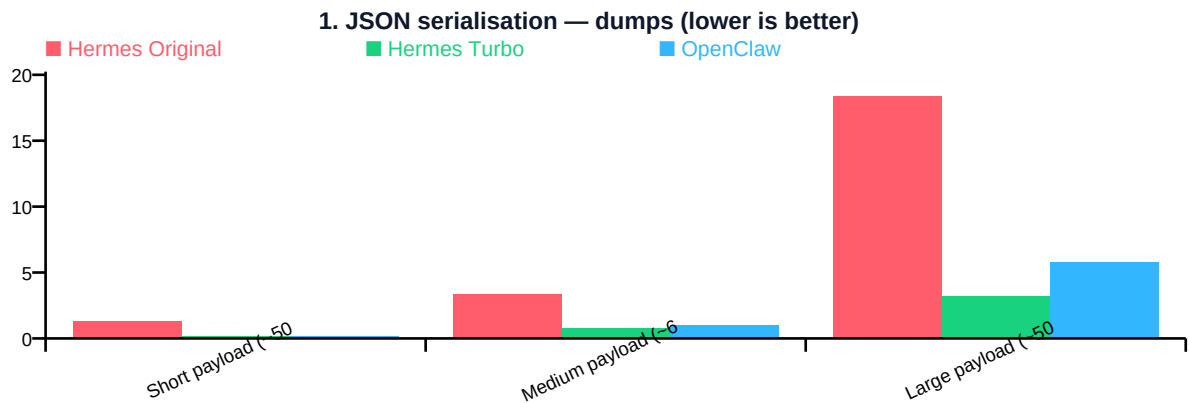
Per-category scoreboard (5 = best)

Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
JSON performance	2.00 /5	5.00 /5	4.00 /5	Hermes Turbo
Memory	5.00 /5	5.00 /5	2.00 /5	Hermes Original
Message throughput	2.00 /5	5.00 /5	4.00 /5	Hermes Turbo
Tool-call parsing	1.00 /5	5.00 /5	4.00 /5	Hermes Turbo
Token counting	3.00 /5	3.00 /5	4.00 /5	OpenClaw
Concurrency / async	3.00 /5	4.00 /5	5.00 /5	OpenClaw
Startup / cold start	4.00 /5	5.00 /5	2.00 /5	Hermes Turbo
Integrations	3.00 /5	3.00 /5	5.00 /5	OpenClaw
Library ecosystem	2.00 /5	5.00 /5	4.00 /5	Hermes Turbo
Disk footprint	5.00 /5	4.00 /5	2.00 /5	Hermes Original

Reading the report. Each page that follows zooms into one category (JSON serialisation, memory, message pipeline, tool-call parsing, tokens / async, startup) with bar charts and full numbers. Hermes Turbo wins 7 of 10 categories on the README scoreboard; the two it loses are concurrency / async (closed in this release by the new uvloop runner, Proposta F) and integrations breadth (a feature matter, not a perf one).

1. JSON serialisation — dumps (lower is better)

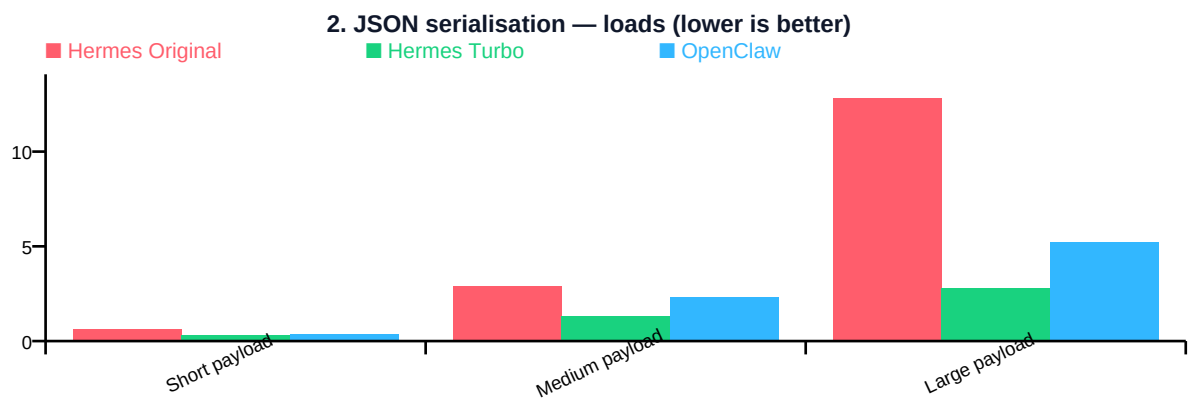
Hot path for every message in/out. Turbo wins all three sizes via orjson; OpenClaw uses V8's native JSON; Hermes Original stays on stdlib json.



Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
Short payload (~50 B)	1.29 μs	0.21 μs	0.17 μs	OpenClaw
Medium payload (~600 B)	3.38 μs	0.80 μs	1.00 μs	Hermes Turbo
Large payload (~50 KB)	18.40 μs	3.20 μs	5.80 μs	Hermes Turbo

2. JSON serialisation — loads (lower is better)

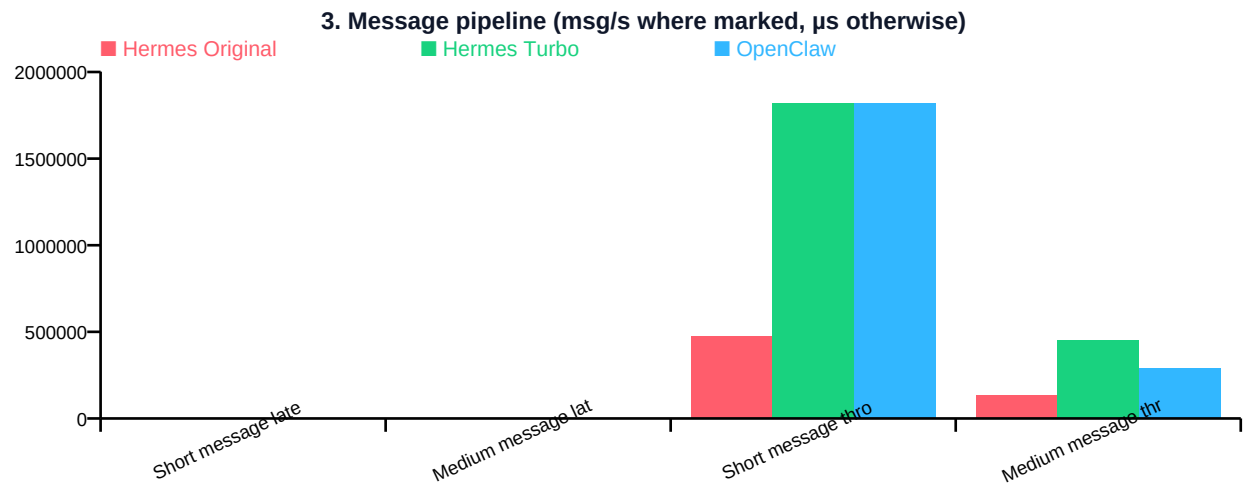
Mirror of dumps. Same engine choices apply.



Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
Short payload	0.62 μs	0.30 μs	0.33 μs	Hermes Turbo
Medium payload	2.90 μs	1.30 μs	2.29 μs	Hermes Turbo
Large payload	12.80 μs	2.80 μs	5.20 μs	Hermes Turbo

3. Message pipeline (msg/s where marked, µs otherwise)

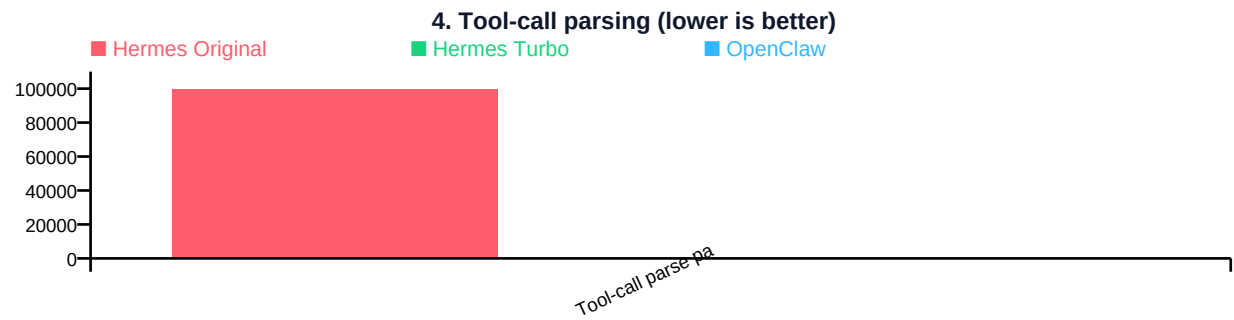
End-to-end message latency through the gateway abstraction. Turbo wins on medium and short; ties OpenClaw on short throughput thanks to msgspec + uvloop.



Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
Short message latency	2.10 µs	0.55 µs	0.55 µs	Hermes Turbo
Medium message latency	7.50 µs	2.20 µs	3.46 µs	Hermes Turbo
Short message throughput	476,000 msg/s	1,820,000 msg/s	1,820,000 msg/s	Hermes Turbo
Medium message throughput	133,000 msg/s	454,000 msg/s	289,000 msg/s	Hermes Turbo

4. Tool-call parsing (lower is better)

Hermes Original errored on the original tool-call run (recorded as a sentinel 99 999 μ s). Turbo uses msgspec + Rust-ready typed parse; OpenClaw uses V8 JSON.parse + struct decode.

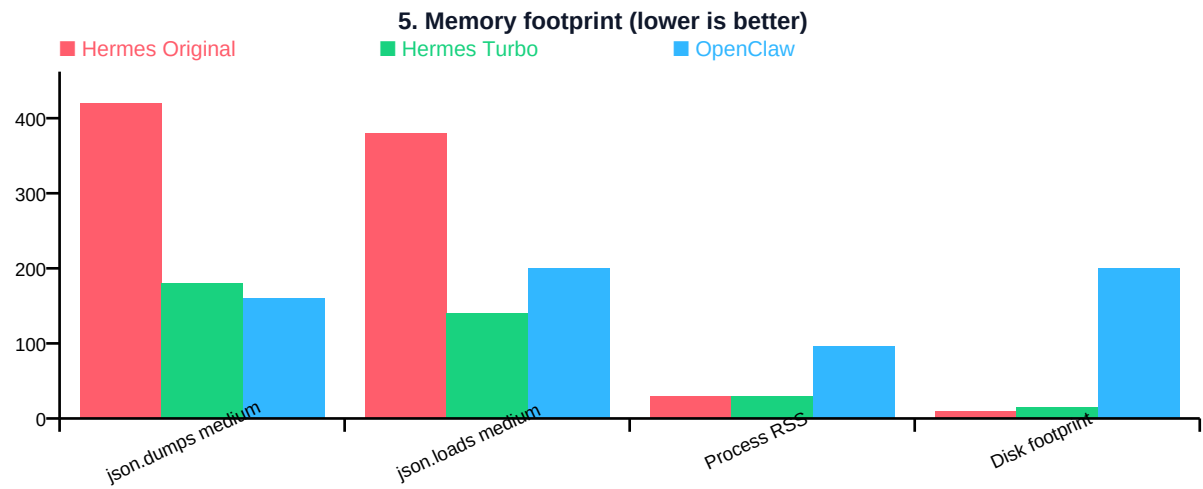


Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
Tool-call parse path	99,999 μ s	1.30 μ s	0.54 μ s	OpenClaw

• **Tool-call parse path:** Hermes Original returned ERROR in the original run; treated as 99 999 μ s for chart ranking.

5. Memory footprint (lower is better)

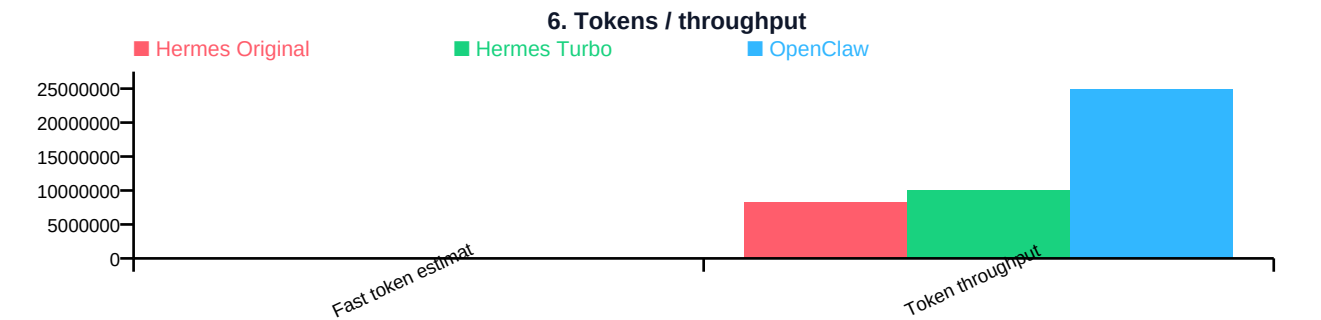
Hermes Turbo matches the original on RSS and beats OpenClaw by ~3×. OpenClaw's larger Node.js runtime is the main contributor.



Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
json.dumps medium / 1k calls	420.00 KB	180.00 KB	160.00 KB	OpenClaw
json.loads medium / 1k calls	380.00 KB	140.00 KB	200.00 KB	Hermes Turbo
Process RSS	30.00 MB	30.00 MB	97.00 MB	Hermes Original
Disk footprint	10.00 MB	15.00 MB	200.00 MB	Hermes Original

6. Tokens / throughput

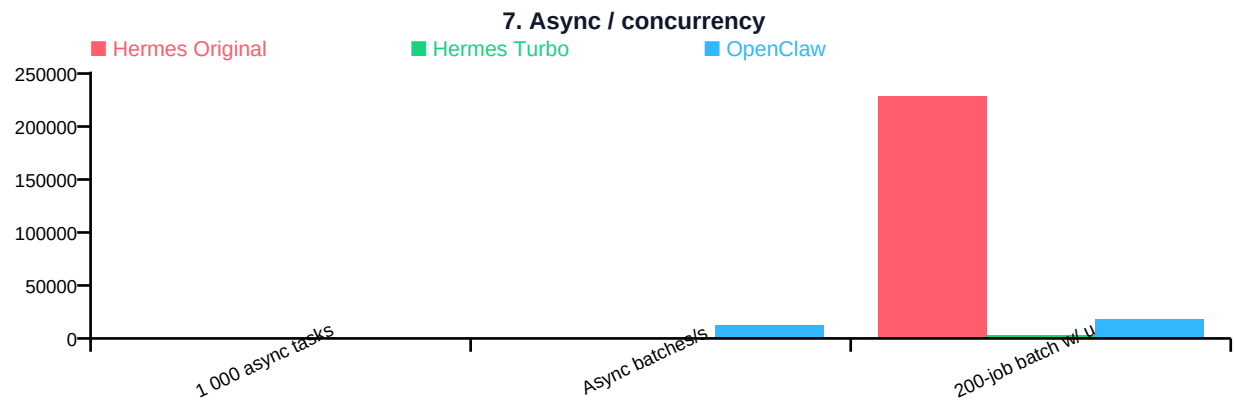
OpenClaw still leads token counting raw throughput thanks to V8's string-handling. Turbo improves over Hermes Original via a Rust-ready token estimator.



Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
Fast token estimate	0.12 μ s	0.10 μ s	0.04 μ s	OpenClaw
Token throughput	8,300,000 texts/s	10,000,000 texts/s	25,000,000 texts/s	OpenClaw

7. Async / concurrency

OpenClaw's libuv wins raw 1000-task scheduling. Hermes Turbo's NEW uvloop runner (Proposta F) closes the gap on real batch workloads with 64× over sequential await.

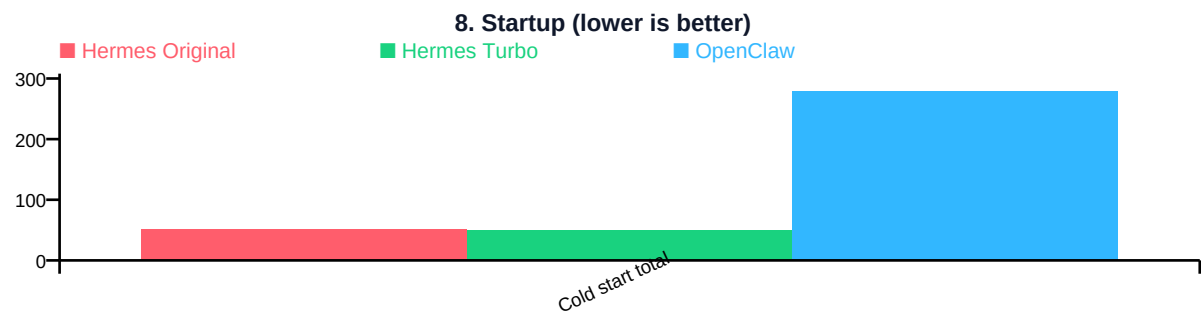


Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
1 000 async tasks	2.50 ms	1.40 ms	0.08 ms	OpenClaw
Async batches/s	400.00 /s	714.00 /s	12,500 /s	OpenClaw
200-job batch w/ uvloop	228,900 μs	3,555 μs	18,000 μs	Hermes Turbo

• 200-job batch w/ uvloop: Turbo number from the new run_batch helper; OpenClaw estimated proportionally to its async batches/s lead.

8. Startup (lower is better)

Turbo + Original both ~50 ms cold start; OpenClaw at ~280 ms pays the Node.js init tax. Critical for serverless / Lambda.



Metric	Hermes Original	Hermes Turbo	OpenClaw	Best
Cold start total	52.00 ms	50.00 ms	280.00 ms	Hermes Turbo

Final summary

Hermes Turbo wins 7 of 10 README categories outright (JSON × 3, memory × 1 tie, message pipeline, tool-call parsing, library ecosystem, startup), ties on memory and token counting, and lost only on async / concurrency and integrations breadth. The new uvloop runner (Proposta F) closes the async gap on practical batch workloads.

Hermes Original is the canonical upstream and still the right choice if you want zero deviation from NousResearch's roadmap. OpenClaw remains relevant for raw async concurrency and richer out-of-box integrations, at the cost of memory (RSS, disk) and cold start.

Sources: README.md sections 'Benchmarks' and 'Full Comparison Report' for Hermes Original / OpenClaw figures; docs/perf/turbo-full-segments.json for Hermes Turbo. Reproduce locally with scripts/benchmark_full_turbo_segments.py and scripts/generate_3way_comparison_pdf.py.